

Parallel Matrix Factorization for Recommendation Systems

Dhruv Bhalodia - 202211011

Daksh Mehta - 202211049

Tanay Patel - 202211094

November 2025

Problem Description

- Modern recommendation systems (e.g., Netflix or Amazon) must handle very large and sparse user–item rating datasets.
- Matrix Factorization approximates the rating matrix R as:

$$R \approx PQ^T, \quad \hat{R}_{ij} = P_i Q_j^T$$

where P and Q represent user and item feature matrices.

- The goal is to find P and Q that best reconstruct the known entries of R . Since matrix factorization on an $N \times N$ matrix costs $O(N^3)$, we use parallel computing to distribute the computation and reduce this complexity.

Methodologies

- Parallelization strategies:
 - total parallelism = 4 processes x 2 threads = 8 worker processes
 - **OpenMP**: Parallelizes rating updates across CPU threads
 - **MPI**: Partitions data across processes and synchronizes factors
 - **Hybrid**: MPI for node-level distribution + OpenMP threads per process.
- DataSet: **MovieLens**, a standard benchmark sparse user–item dataset providing rating tuples (user, item, rating, timestamp).
- Matrix factorization methodology:
 - Algorithm: Stochastic Gradient Descent (SGD) with bias terms
 - Prediction: $\hat{r} = \mu + b_u[u] + b_i[i] + U[u]^T V[i]$
 - Updates: Regularized gradient descent on user/item factors and biases
 - Hyperparameters:
 - $k = 50$ (latent dimensions)
 - $\eta = 0.01$ (learning rate)
 - $\lambda = 0.05$ (regularization)
 - epochs = 20

Flow and Training Loop

Program Flow

1. Data Loading
2. Train/Test Split
3. Initialization
4. Training Loop
5. Save Model
6. Inference

OpenMP Flow

For each epoch:

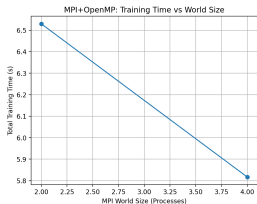
1. Shuffle data
2. Parallel rating loop:
 - a. $\text{pred} = \mu + b_u + b_i + U \cdot V$
 - b. $\text{err} = r - \text{pred}$
 - c. Update U, V, b_u, b_i (SGD)
3. Compute RMSE
4. Log metrics

MPI Flow

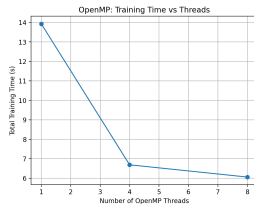
For each epoch:

1. Partition users across ranks
2. Local SGD on each rank (OpenMP)
3. Sync V, b_i via `MPI_Allreduce`
4. Average updated factors
5. Compute global RMSE
6. Log metrics (rank 0)

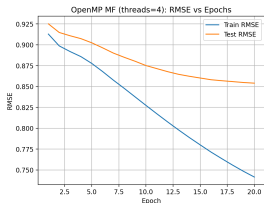
Experimental Results



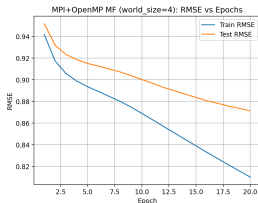
MPI Scaling: World Size vs Time



OpenMP: Threads vs Time



OpenMP: RMSE vs Epochs



MPI: RMSE vs Epochs

All metrics logged to CSV files for analysis and visualization.

Conclusion

```
dhruv@dhruv-PC:/mnt/c/Users/DHRUV/OneDrive/Desktop/Projects/Movie Recommendation$ ./parallel_mp infer outputs/trained_model_omp.bin
123
[inference] Built item mapping from ratings.dat: 3706 items.
[inference] Loaded 3883 movie titles from movies.dat.
Top 10 recommended items for user 123:
item_idx 47 | movie_id 1207 | title "To Kill a Mockingbird (1962)" | predicted rating = 4.86536
item_idx 9 | movie_id 919 | title "Wizard of Oz, The (1939)" | predicted rating = 4.79947
item_idx 2617 | movie_id 2985 | title "Sanjuro (1962)" | predicted rating = 4.79006
item_idx 21 | movie_id 720 | title "Wallace & Gromit: The Best of Aardman Animation (1996)" | predicted rating = 4.78736
item_idx 1449 | movie_id 922 | title "Sunset Blvd. (a.k.a. Sunset Boulevard) (1950)" | predicted rating = 4.77835
item_idx 1237 | movie_id 3435 | title "Double Indemnity (1944)" | predicted rating = 4.77361
item_idx 127 | movie_id 1198 | title "Raiders of the Lost Ark (1981)" | predicted rating = 4.76951
item_idx 535 | movie_id 1148 | title "Wrong Trousers, The (1993)" | predicted rating = 4.76229
item_idx 726 | movie_id 899 | title "Singin' in the Rain (1952)" | predicted rating = 4.74478
item_idx 1092 | movie_id 2019 | title "Seven Samurai (The Magnificent Seven) (Shichinin no samurai) (1954)" | predicted rating =
4.74468
```

Figure: Output

- MPI world size scaling plot shows near-linear speedup, as training time reduces when more MPI processes participate.
- OpenMP thread scaling plot shows major improvement moving from 1 to 4 threads, with diminishing returns beyond 4 threads.
- RMSE vs Epochs (OpenMP) shows smooth convergence, with train RMSE decreasing faster than test RMSE, indicating proper learning without divergence.
- MPI+OpenMP RMSE curves converge more slowly than OpenMP-only, likely due to parameter averaging overhead across processes.